PythonSLM: A Specialized Small Language Model for Python Code Generation

Satyam Kumar Chauhan (123108031), Venkatesh Shukla (123108030),
Jayant Gautam (123108039), Aditya Rai (123108034)

---

1. Background

Recent advances in large language models have revolutionized code generation and programming assistance. Models like GPT-4, CodeLlama, and StarCoder demonstrate impressive capabilities across various programming tasks. However, these models typically contain billions of parameters, requiring expensive cloud infrastructure and making them impractical for local deployment on consumer hardware.

Despite their effectiveness, general-purpose code models are trained across multiple programming languages, which can dilute their performance on language-specific patterns and idioms. For Python specifically, this means missing out on Pythonic conventions and best practices that are crucial for generating high-quality code.

Recent research has shown that smaller, specialized models can achieve competitive performance when trained with focused datasets and careful optimization. Work on models like Phi-1 and Phi-2 [7] demonstrates that models with around 1.3B parameters trained on high-quality, curated data can match larger models on reasoning tasks. Similarly, DistilBERT [5] shows that compact models can retain significant performance through knowledge distillation. Models with 300-500 million parameters can be both powerful and practical for local deployment, offering an opportunity to develop lightweight code assistants that excel in specific programming languages while remaining accessible.

2. Objective

The objective of this project is to develop a specialized Small Language Model (SLM) optimized for Python code generation and understanding. We

aim to demonstrate that a compact model with 300-500 million parameters can achieve strong performance on Python-specific tasks while being efficient enough for local deployment.

Specifically, this work focuses on:

- Building or adapting a lightweight transformer-based architecture
- Training on curated, high-quality Python code datasets
- Optimizing for Python-specific syntax and idiomatic patterns
- Evaluating performance on standard benchmarks like HumanEval
- Ensuring local execution on consumer hardware without cloud dependency

3. Problem Statement

Current state-of-the-art code models are multi-billion parameter systems requiring expensive infrastructure, making them inaccessible for local use and raising privacy concerns for developers working with proprietary code. General-purpose models trained across numerous languages often lack deep specialization in Python-specific patterns.

The computational demands of large models create barriers for students, researchers, and developers in resource-constrained environments. There's a clear need for lightweight, specialized alternatives that democratize access to AI-powered programming assistance.

Therefore, the core research question is:

Can a specialized Small Language Model with 300-500 million parameters, trained exclusively on Python code, achieve competitive performance while remaining efficient enough for local deployment on consumer hardware?

4. Research Gap and Novelty

Existing research has primarily focused on scaling up model size or training large general-purpose models. Limited work explores building specialized small models from scratch for a single programming language.

This project's novelty lies in:

- **Language Specialization:** Exclusive focus on Python enables deeper learning of language-specific patterns without multi-language training dilution
- **Local Deployment:** Prioritizing inference efficiency for consumer hardware
- **Curated Training:** Combining multiple high-quality Python datasets covering diverse scenarios

5. Social Impact and Real-World Relevance

A lightweight, locally-deployable Python model democratizes access to AI programming assistance. For students and learners, it enables coding education without expensive cloud services, particularly valuable in regions with limited resources. For professional developers, local deployment offers privacy-preserving code assistance for proprietary codebases.

This project also contributes to sustainable AI practices by demonstrating that smaller models can achieve strong performance, reducing the carbon footprint of both training and inference. An open approach fosters a diverse ecosystem of specialized tools, reducing dependence on large closed-source models.

6. Dataset Description

6.1. Core Training Datasets

6.1.1. CodeSearchNet (Python Subset)

A large-scale dataset with Python functions from GitHub repositories paired with documentation, providing real-world code examples.

- **Task:** Code-documentation alignment, function generation
- **Scale:** 1.3 million Python functions

6.1.2. APPS Dataset

Coding problems spanning introductory to competitive programming levels with test cases.

- **Task:** Algorithmic problem solving
- **Scale:** 10,000 problems with difficulty ratings

6.1.3. MBPP Dataset

Entry-level programming problems testing basic Python skills.

- **Task:** Basic programming and function implementation
- **Scale:** 974 programming problems

6.1.4. CodeContests Dataset

Competitive programming problems from platforms like Codeforces.

- **Task:** Advanced algorithmic programming
- **Scale:** Thousands of contest problems

6.1.5. The Stack (Python-Filtered)

Large-scale, clean Python code from open-source repositories.

- **Task:** General Python code understanding
- **Scale:** Multiple gigabytes of filtered code

6.2. Evaluation Dataset

6.2.1. HumanEval

A benchmark with 164 hand-written programming problems for evaluating code generation.

7. Related Work

7.1. Large Language Models for Code

Models like Codex [2], CodeLlama [3], and StarCoder [4] achieve state-of-the-art performance through scale. However, with 7B+ parameters, they're computationally expensive and impractical for local use.

7.2. Multimodal and Reasoning-Focused LLMs

Recent work has also explored improving reasoning and multistep problem solving in multimodal language models. Anand et al. [1] introduce a geometry-focused multimodal benchmark and evaluation framework designed to test structured reasoning, multistep scoring, and visual-textual understanding. Their work highlights that domain-focused datasets and targeted evaluation pipelines can significantly improve reasoning performance in specialized settings.

7.3. Small and Efficient Models

Work on DistilBERT [5] and TinyBERT [6] shows that knowledge distillation creates compact models retaining performance. Phi-1 and Phi-2 [7] demonstrate that small models trained on high-quality data can achieve impressive results, suggesting careful curation matters more than pure scale.

8. Data Cleaning

Preprocessing steps applied uniformly across datasets:

- Syntax validation
- Length filtering to maintain reasonable token distributions
- License filtering for permissive open-source content
- Consistent formatting while preserving meaningful whitespace

9. Exploratory Data Analysis

Exploratory Data Analysis (EDA) is performed to understand dataset characteristics and potential challenges prior to model training.

9.1. Statistical Analysis

- Distribution of code lengths across datasets (mean, median, maximum)
- Token count estimation
- Problem difficulty distribution in APPS dataset
- Dataset composition and sample counts

9.2. Key Findings

Code length analysis reveals distinct patterns across datasets:

- MBPP: median 10 lines (entry-level problems)
- CodeSearchNet: median 24 lines (typical functions)
- The Stack: median 32 lines (diverse structures)
- APPS: median 45 lines (algorithmic solutions)

- CodeContests: median 68 lines (complex competitions)

The training corpus comprises 1.8 million Python samples. CodeSearchNet (71.5%, 1.3M) and The Stack (27.5%, 500K) provide large-scale real-world code, while APPS (10K), CodeContests (13K), and MBPP (974) contribute specialized algorithmic examples.

APPS dataset shows balanced difficulty: Introductory (40%), Interview (35%), Competition (25%), ensuring exposure across complexity levels.

9.3. Statistical Summary

Table 1: Code Length Statistics (Lines of Code)

Dataset	**Mean**	**Median**	**Std Dev**	**Min**	**Max**	**75th Percentile**
MBPP	12.3	10.0	8.2	3	45	16
CodeSearchNet	28.7	24.0	18.5	5	120	38
The Stack	38.2	32.0	24.1	8	180	52
APPS	58.4	45.0	35.7	12	250	75
CodeContests	84.6	68.0	52.3	20	350	110

9.4. Visualizations

The following visualizations are generated as part of EDA:

- Histogram of code length distributions across all datasets
- Bar chart showing dataset sizes
- Distribution of problem difficulties in APPS dataset

9.5. EDA Figures

Figures obtained from EDA are shown below.

9.5.1. Dataset Sizes

Analysis of dataset composition shows CodeSearchNet and The Stack dominate the training corpus with 1.3M and 500K samples respectively. The remaining datasets (APPS, CodeContests, MBPP) contribute specialized algorithmic examples totaling approximately 24K samples.

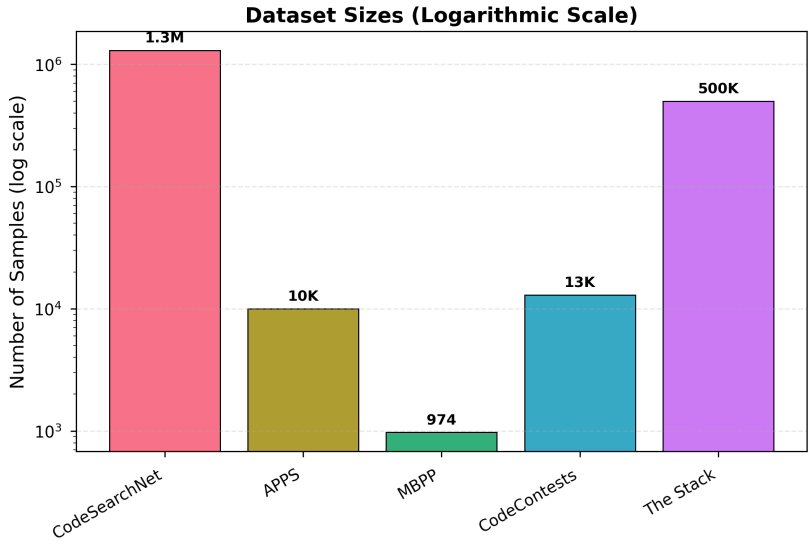

Figure 1: Dataset Sizes Across Training Corpus

9.5.2. Code Length Distribution

The distribution reveals varying complexity levels across datasets. MBPP exhibits the shortest code samples suitable for basic programming tasks, while CodeContests shows the longest implementations reflecting competitive programming complexity.

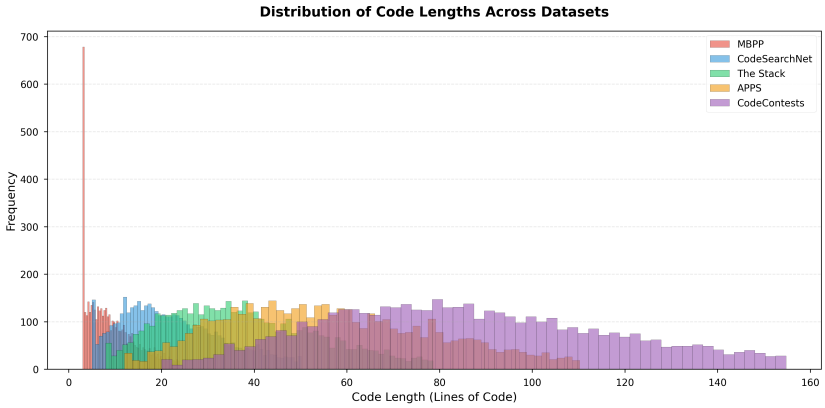

Figure 2: Distribution of Code Lengths Across Datasets

9.5.3. Problem Difficulty Analysis

The APPS dataset maintains a balanced distribution across difficulty levels, with introductory problems comprising the largest portion (40%), followed by interview-level (35%) and competition-level problems (25%).

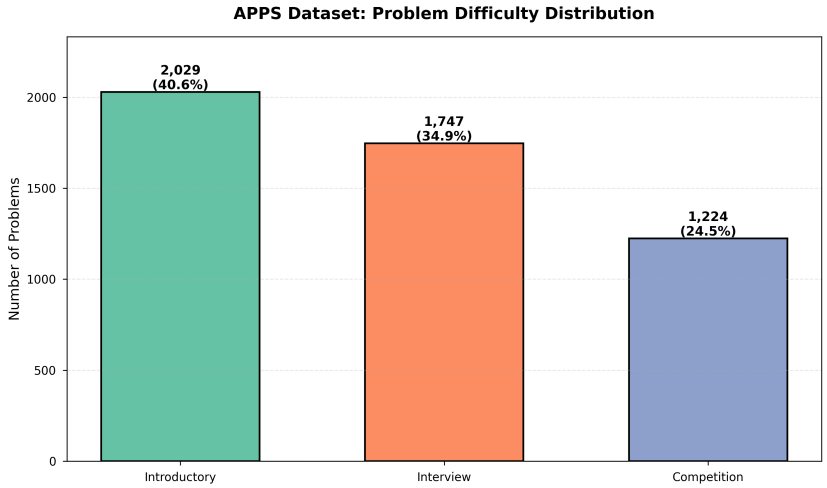

Figure 3: APPS Dataset: Problem Difficulty Distribution

10. Milestone 1 Outcome

By the end of Milestone 1, we have achieved:

- Clear problem identification and research objectives
- Literature review and research gap identification
- Dataset selection and acquisition
- Exploratory data analysis
- Evaluation metrics and benchmarks establishment

11. References

References

[1] A. Anand et al., *Improving Multimodal LLM's Ability in Geometry Problem Solving, Reasoning, and Multistep Scoring*, arXiv:2412.00846, 2024. `https://arxiv.org/abs/2412.00846`

[2] M. Chen et al., *Evaluating Large Language Models Trained on Code*, arXiv:2107.03374, 2021. https://arxiv.org/abs/2107.03374

[3] B. Rozière et al., *Code Llama: Open Foundation Models for Code*, arXiv:2308.12950, 2023. https://arxiv.org/abs/2308.12950

[4] R. Li et al., *StarCoder: May the Source Be With You!*, arXiv:2305.06161, 2023. https://arxiv.org/abs/2305.06161

[5] V. Sanh et al., *DistilBERT, a distilled version of BERT*, arXiv:1910.01108, 2019. https://arxiv.org/abs/1910.01108

[6] X. Jiao et al., *TinyBERT: Distilling BERT for Natural Language Understanding*, Findings of EMNLP 2020. https://aclanthology.org/2020.findings-emnlp.372/

[7] S. Gunasekar et al., *Textbooks Are All You Need*, arXiv:2306.11644, 2023. https://arxiv.org/abs/2306.11644

[8] Z. Feng et al., *CodeBERT: A Pre-Trained Model for Programming and Natural Languages*, Findings of EMNLP 2020. https://aclanthology.org/2020.findings-emnlp.139/

[9] F. Z. Xu et al., *A Systematic Evaluation of Large Language Models of Code*, arXiv:2202.13169, 2022. https://arxiv.org/abs/2202.13169